

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. Why is

try:

```
print(10 / 0)
```

except Exception:

```
print("A")
```

except ZeroDivisionError:

```
print("B")
```

codes's second handler never reached?

A)ZeroDivisionError occurs before Exception

B)Exception is a superclass of ZeroDivisionError

C)ZeroDivisionError is not an exception

D)except blocks execute randomly

Q2. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError

B)Module loaded

10

C)10

10

D)Module loaded

Module loaded

10

Q3. What is the output?

try:

```
x = int("abc")
```

except ValueError:

```
print("A")
```

else:

```
print("B")
```

finally:

```
print("C")
```

A)A

B)B C

C)B

D)A C

Q4. What will be the output?

def fun(x):

try:

if x < 0:

```
raise ValueError("negative")
```

```
return x
```

except ValueError:

```
return 100
```

finally:

```
print("cleanup")
```

```
print(fun(-5))
```

A)negative

cleanup

B)cleanup

-5

C)cleanup

100

D)100

cleanup

Q5. What happens here?

class MyError(Exception):

```
pass
```

def test():

```
raise MyError("Something went wrong")
```

try:



```
test()
except MyError as e:
    print(type(e).__name__)
```

A)MyError

B)TypeError

C)Exception

D)Something went wrong

Q6. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

A)5

B)10

C>Error

D)func(5)

Q7. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6

B)36

C)NameError

D)6.0

Q8. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

A)x

display

B)Only Hello

C>Error

D)10

Hello

Q9. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)5

B)6

C)3

D)None

Q10. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)24

B)10

C)0

D)9

Q11. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

A)0

B)TypeError

C)NameError

D)None

Q12. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?



- A)B
- B)A and B
- C)Nothing
- D)A

Q13. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

fun()

- A)20
- B)UnboundLocalError
- C)NameError
- D)10

Q14. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)25
- B)5
- C)sqrt(25)
- D)5.0

Q15. Which object keeps track of modules that have already been imported during the current Python process?

- A)__module__
- B)sys.path
- C)sys.imports
- D)sys.modules

Q16. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)10
- B)15
- C>Error
- D)5

Q17. What will be printed?

```
def make_counter():
    count = 0
```

```
def counter():
    nonlocal count
    count += 1
    return count
```

```
return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)1

1

1

1

B)0

1

0

2

C)1

2

3

4

D)1

2

1

3

Q18. What is the output?

```
def outer():
    x = 10
```

```
def inner():
    nonlocal x
    x = x + 5
    return x
```

```
inner()
return x
```

```
print(outer())
```

A)5

B)15

C)10

D)UnboundLocalError



Q19. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)30
- B)20
- C)10
- D)-10

Q20. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)First imports only sqrt
- B)There is no difference
- C)First provides the module namespace; second imports sqrt directly
- D)Both import only sqrt

Q21. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)NameError
- B)UnboundLocalError
- C)50
- D)10

Q22. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

- A)SyntaxError
- B)B
- C)A B
- D)A

Q23. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[2, 4]
- B)[1, 3, 5]
- C)[2, 4, 6, 8, 10]
- D)[4, 8]

Q24. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)
```

- A)15 5
- B)15 15
- C)5 5
- D)5 15

Q25. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

- A)No Output
- B)A, followed by propagation of the same exception
- C)B
- D)A, and execution ends normally

Q26. if test.py has

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```



If test.py is executed directly, what will be printed?

- A)A followed by B
- B)B
- C)Nothing
- D)A

Q27. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

- A)TypeError
- B)6
- C)3
- D)RecursionError

Q28. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents syntax errors
- B)It identifies code intended to run when the file is executed directly
- C)It creates the main function
- D)It prevents all imports

Q29. What happens?

```
def fun(a, b):  
    return a + b
```

print(fun(10, a=20))

- A)30
- B)TypeError
- C)20
- D)SyntaxError

Q30. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")
```

```
    return 20  
finally:  
    print("D")
```

print(fun())

- A)A
- B
- D
- 10
- B)A
- B
- 10
- D
- C)A
- B
- C
- D
- 10
- D)A
- C
- D
- 20

Q31. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

- A)InvalidAgeError
- Age is too low
- B)Exception
- Age is too low
- C)Nothing, because custom exceptions cannot be caught



D)InvalidAgeError

15

Q32. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

A)NameError

B)RuntimeError cannot propagate through functions

C)TypeError

D)Handled

Q33. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)TypeError

B)SyntaxError

C)20

D)30

Q34. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)NameError

B)None

C)10

D)20

Q35. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B C

C)A C

D)B

Q36. What will normally happen?

```
import math
```

```
import math
```

A)SyntaxError

B)ImportError

C)The module is normally loaded once and reused

D)The module is normally loaded and executed twice

Q37. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

A)NameError

B)TypeError

C)It executes correctly

D)SyntaxError

Q38. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)10

B)NameError

C)15

D)None



Q39. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

- A)A
- B)Nothing
- C)A B
- D)B

Q40. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)ValueError
- B)e
- C)Wrong Value
- D)Nothing

Q41. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)Nothing
- B)A B
- C)A
- D)B

Q42. What will be the output?

x = 10

```
def outer():  
    x = 20  
  
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

**outer()
print(x)**

- A)20
- 10
- B)30
- 20
- C)20
- 30
- D)30
- 30

Q43. What does the following function return?

```
def fun():  
    print("Hello")
```

x = fun()

print(x)

- A>Error
- B>Hello
- C>Hello None
- D)None

Q44. What will be printed?

x = 10

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

outer()

- A)10
- B)30
- C)20
- D)NameError

Q45. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

a = fun()



b = fun()

print(a)

print(b)

A)[10]

[10, 10]

B)[10]

[10]

C)[10, 10]

[10]

D)Error

Q46. What will happen?

def fun(a, b=10):

return a + b

print(fun(b=5, a=20))

A)SyntaxError

B)15

C)TypeError

D)25

Q47. What will happen?

def outer():

x = 10

def inner():

nonlocal y

y = 20

inner()

outer()

A)SyntaxError

B)10

C)NameError

D)20

Q48. What will be printed?

def fun():

try:

return 10

finally:

print("Finally")

print(fun())

A)10 followed by Finally

B)Finally followed by 10

C)10 only

D)Finally only

Q49. What is the output?

def fun():

try:

return 10

finally:

return 20

print(fun())

A)RuntimeError

B)None

C)20

D)10

Q50. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)The local file can shadow the intended standard-library module

C)Python will always reject it

D)Python automatically renames the local file

